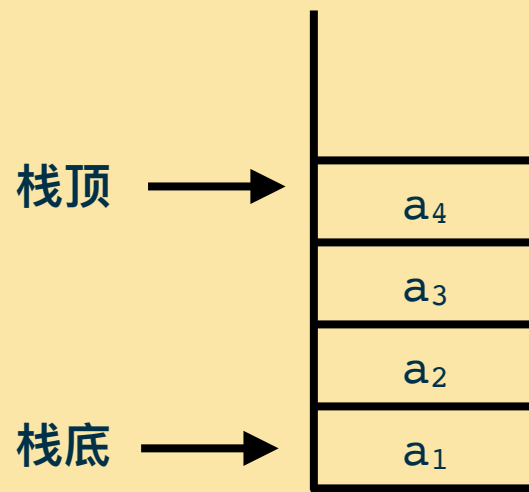




1. 栈的定义和基本操作

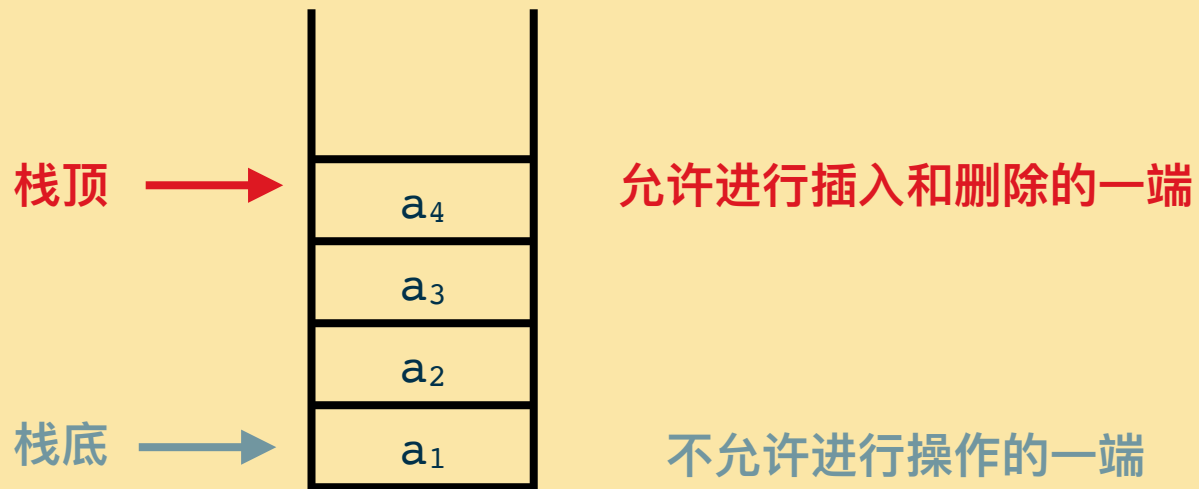


栈

只能在一端进行插入和删除的线性表



1. 栈的定义和基本操作

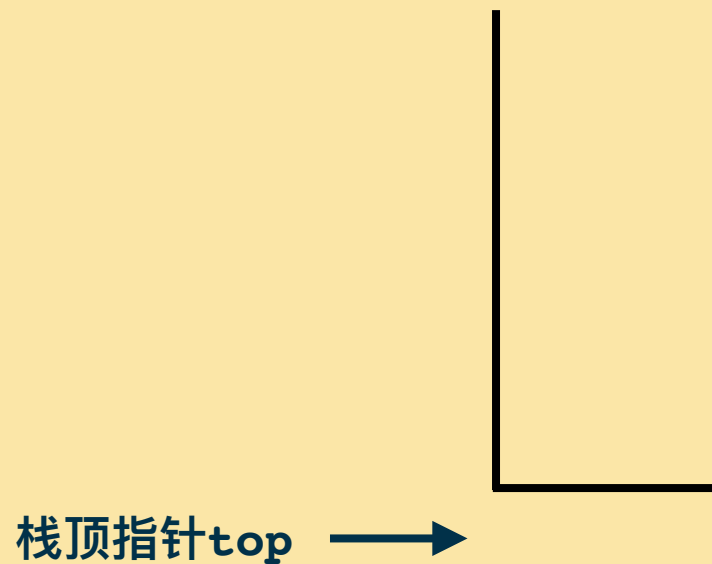


栈

只能在一端进行插入和删除的线性表



1. 栈的定义和基本操作



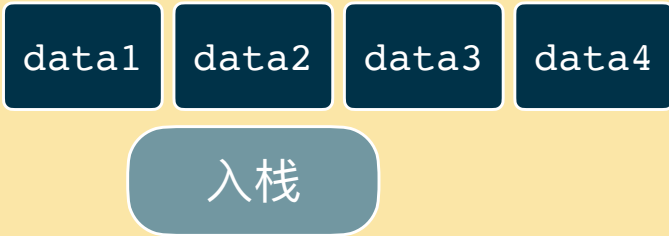
栈的操作特性：**先进后出** (Last In First out, LIFO)

栈 只能在一端进行插入和删除的线性表



1. 栈的定义和基本操作

入栈1, 2, 3, 4, 再依次出栈



栈顶指针top →

栈的操作特性: **先进后出** (Last In First out, LIFO)

栈 只能在一端进行插入和删除的线性表



1. 栈的定义和基本操作

入栈1, 2, 3, 4, 再依次出栈

data4

data3

data2

data1

出栈

对于同一个入栈序列，如果我们规定的操作序列不同，那么出栈序列也会不同
所以，在没有限定操作序列的前提下，入栈序列和出栈序列会呈现一对多的关系

栈顶指针top →

栈的操作特性：**先进后出** (Last In First out, LIFO)

栈 只能在一端进行插入和删除的线性表



1. 栈的定义和基本操作

ADT Stack{

数据对象 : $D=\{a_i | a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0\}$

数据关系 : $R=\{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2, \dots, n \}$

约定 a_n 端为栈顶, a_1 端为栈底。

基本操作 :

InitStack(&S)

操作结果 : 构造一个空栈S。

DestroyStack(&S)

初始条件 : 栈S已存在。

操作结果 : 栈S被销毁。

ClearStack(&S)

初始条件 : 栈S已存在。

操作结果 : 将S清为空栈。

StackEmpty(S)

初始条件 : 栈S已存在。

操作结果 : 若栈S为空栈, 则返回true, 否则返回false。

StackLength(S)

初始条件 : 栈S已存在。

操作结果 : 返回S的元素个数, 即栈的长度。

GetTop(S)

初始条件 : 栈S已存在且非空。

操作结果 : 返回S的栈顶元素, 不修改栈顶指针。

Push(&S, e)

初始条件 : 栈S已存在。

操作结果 : 插入元素e为新的栈顶元素。

Pop(&S, &e)

初始条件 : 栈S已存在且非空。

操作结果 : 删除S的栈顶元素, 并用e返回其值。

StackTraverse(S)

初始条件 : 栈S已存在且非空。

操作结果 : 从栈底到栈顶依次对S的每个数据元素进行访问。

}ADT Stack



1. 栈的定义和基本操作

栈的基本操作

InitStack (&S) : 初始化一个空栈。

StackEmpty (S) : 判断栈是否空。若栈S为空, 返回true; 否则返回false

Push (&S, x) : 入栈操作。若栈未满, 则x成为新的栈顶元素

Pop (&S, &x) : 出栈操作。若栈非空, 则弹出栈顶元素, 并通过x返回该值

GetTop (S, &x) : 读取栈项元素但不出栈。若栈非空, 则通过x返回栈项元素

Destroystack (&S) : 销毁栈S, 并释放其所占用的存储空间



2. 栈的顺序表示

①顺序栈的定义

```
#define MaxSize 50 //定义顺序栈的最大容量为 50
typedef struct{
    ElemType data[MaxSize]; //用数组 data 存放栈中的元素
    int top; //栈顶指针，记录当前栈顶元素的位置
}SqStack; //顺序栈的结构类型为SqStack
```



2. 栈的顺序表示

②顺序栈的基本操作

初始化

```
void InitStack(SqStack &S){  
    S.top = -1; //将栈顶指针设为 -1，表示当前栈为空  
}
```

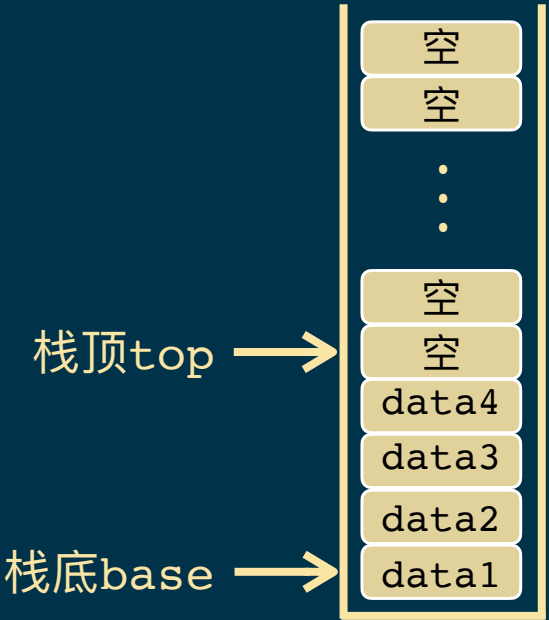


2. 栈的顺序表示

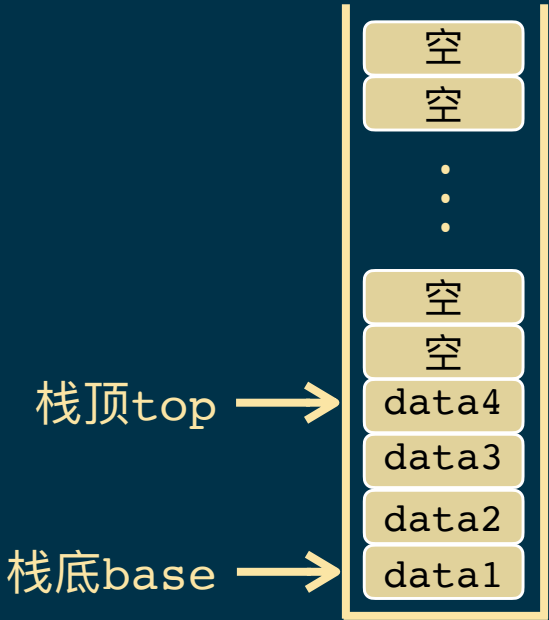
②顺序栈的基本操作

指针设置

栈顶指针指向栈顶元素后一位置



栈顶指针指向栈顶元素





2. 栈的顺序表示

②顺序栈的基本操作

判栈空

```
bool StackEmpty(SqStack S){  
    if(S.top == -1) //栈顶指针为-1, 则栈空  
        return true;  
    else return false; //否则返回false, 表示栈非空  
}
```



2. 栈的顺序表示

②顺序栈的基本操作

入栈

```
bool Push(SqStack &S, ElemType x){  
    if(S.top == MaxSize-1) //栈满, 报错  
        return false;  
    S.data[++S.top] = x; //栈顶指针先加1, 再把x存入新的栈顶位置  
    return true; //入栈成功, 返回true  
}
```



2. 栈的顺序表示

②顺序栈的基本操作

出栈

```
bool Pop(SqStack &S, ElemType &x){  
    if(S.top == -1) //栈空, 报错  
        return false;  
    x = S.data[S.top--]; //先将栈顶元素读取到x中, 再把栈顶指针减1  
    return true; //出栈成功, 返回true  
}
```



2. 栈的顺序表示

②顺序栈的基本操作

读栈顶元素

```
bool GetTop(SqStack S, ElemType &x){  
    if(S.top == -1) //栈空, 报错  
        return false;  
    x = S.data[S.top]; //将栈顶元素读取到x中  
    return true;  
}
```



3. 栈的链式表示

①链栈的定义

```
Typedef struct StackNode{ //定义链栈中节点的结构体类型
    ElemType data; //data用来存放当前节点的数据元素
    struct StackNode *next; //指针next指向下一个节点
}StackNode, *LinkStack;
//StackNode是节点类型, LinkStack是指向节点的指针类型
```



3. 栈的链式表示

②链栈的基本操作

初始化

```
bool InitStack(LinkStack &S){  
    S = NULL; //令栈顶指针为空，表示这是一个空栈  
    return true; //初始化成功，返回true  
}
```



3. 栈的链式表示

②链栈的基本操作

入栈

```
bool Push(LinkStack &S, ElemType e){  
    p = (StackNode*)malloc(sizeof(StackNode)); //为新节点申请一块存储空间  
    p->data = e; //将入栈元素e存入新节点的数据域  
    p->next = S; //让新节点的next指向原来的栈顶节点  
    S = p; //更新栈顶指针s，使其指向新的栈顶节点  
    return true; //入栈成功，返回true  
}
```



3. 栈的链式表示

②链栈的基本操作

出栈

```
bool Pop(LinkStack &S, ElemType &e){  
    e = S->data; //先取出栈顶元素，保存到 e 中  
    p = S; //用指针p暂存当前栈顶元素  
    S = S->next; //栈顶指针后移，指向原栈顶节点的下一个节点  
    free(p); //释放原栈顶节点所占的存储空间  
    return true; //出栈成功，返回true  
}
```



3. 栈的链式表示

②链栈的基本操作

读栈顶元素

```
ElemType GetTop(LinkStack S){  
    if(S != NULL) return S->data; //若栈非空，则返回栈顶元素  
}
```



4. 卡特兰数

有n个元素入栈，那么有多少种不同的出栈方式？

$$\text{卡特兰数} \frac{1}{n+1} C_{2n}^n$$



4. 卡特兰数

有n个元素入栈，那么有多少种不同的出栈方式？

$$\text{卡特兰数} = \frac{1}{n+1} C_{2n}^n$$

$$\begin{aligned} \text{PS: } C_{2n}^n &= \frac{A_{2n}^n}{n!} \\ \text{如 } C_6^3 &= \frac{6 \times 5 \times 4}{3 \times 2 \times 1} \\ C_8^4 &= \frac{8 \times 7 \times 6 \times 5}{4 \times 3 \times 2 \times 1} \end{aligned}$$

祝你今天特别特别开心！